



Hands-on assignment 4

2026. 6. 2. 21:05 · 카테고리 없음 수정 삭제 비공개

목차



Problem definition



Step1:



방법1) $2n-2$ 번의 비교가 있는 경우



방법 2) $3n-2\frac{3n}{2} - 2$ 비교가 있는 경우



Step2:



Step3:



- Course: Computer Algorithm
- StudentID: 2466044
- Author: Sieun Lee
- Date: 2026-06-03
- Repository(public): <https://github.com/sihyes/2026-ComputerAlgorithm/tree/main/Assignment04>

Hands-on assignment 4-1: findMaxMin

Do NOT use AI for this HA4!!

Problem definition: We have an unsorted array [24,75,92,83,61,48,97,50]. Let's find MAX and MIN using your findMaxMin algorithm

- Step1: Implement two types of findMaxMin algorithm (Refer to pseudo code)

- Step1: Implement two types of findMaxMin algorithm (Refer to pseudo code in slide 33)

- First type should yield $2n-2$ comparison
- First type should yield $\frac{3n}{2} - 2$ comparisons
- Step2: Analyze the time complexities of your implementations
 - Sufficient details have to be provided based on your code and time complexity (This complexity analysis just shows the number of comparisons based on your code, NOT use recurrence relations and asymptotic bound)
- Step3: Explain why one type performs more efficiently than the other

문제 전문

Problem definition

We have an unsorted array [24,75,92,83,61,48,97,50]. Let 's find MAX and MIN using your findMaxMin algorithm

배열 A = [24,75,92,83,61,48,97,50] 라고 가정하자, 이 A에서 최댓값과 최솟값을 찾는다.

Step1:

Implement two types of findMaxMin algorithm (Refer to pseudo code in slide 33)

- First type should yield $2n-2$ comparison
- First type should yield $3n-2$ comparisons

방법1) $2n-2$ 번의 비교가 있는 경우

이 방법은 최솟값과 최댓값을 각각 별도로 구한다. linear한 방식으로 진행된다.

pseudo code



```

1 procedure FindMaxMin_Naive(A)
2
3 1. MIN ← A[0]
4 2. MAX ← A[0]
5
6 3. for i ← 1 to length(A)-1 do
7
8 4.     if A[i] > MAX then
9 5.         MAX ← A[i]
10
11 6.     if A[i] < MIN then
12 7.         MIN ← A[i]
13
14 8. return (MIN, MAX)

```

python

전체적인 로직은 다음과 같다.

1. `max` 와 `min` 을 첫 번째 element인 `A[0]=24`로 초기화한다.
2. 배열의 두 번째 element부터 마지막까지 순차적으로 3번 내용을 처리한다.
3. 각 element에 대해 두 가지 비교를 수행한다.
 - 3.1. 이 element가 기존 max보다 큰지 (크다면 update)
 - 3.2 이 element가 기존 min보다 작은지 (작다면 update)

```

1 def findMaxMin_Naive(A):
2     n = len(A)
3     max_val = min_val = A[0]
4     for i in range(1, n):
5         if A[i] > max_val:
6             max_val = A[i]
7         if A[i] < min_val:
8             min_val = A[i]
9     return min_val, max_val

```

python

$n = 8$ 일 때 총 $(2 \times 8) - 2 = 14$ 번의 비교가 이루어졌다.

//비교횟수 추가한 버전

```
1 A = [24, 75, 92, 83, 61, 48, 97, 50]
2
3 def findMaxMin_1(A):
4     count = 0
5     n = len(A)
6     max_val = min_val = A[0]
7     for i in range(1, n):
8         count += 1
9         if A[i] > max_val:
10             max_val = A[i]
11
12         count += 1
13         if A[i] < min_val:
14             min_val = A[i]
15
16     return min_val, max_val, count
17
18 min_val_1, max_val_1, count1 = findMaxMin_1(A)
19 print(f"방법 1 - 최솟값: {min_val_1}, 최댓값: {max_val_1}, 비교 횟수: {count1}")
```

python

```
82106@DESKTOP-U8T6R2I MINGW64 ~/develop
```

```
$ C:/Users/82106/AppData/Local/Programs/Python/P
/python.exe c:/Users/82106/develop/2026-Computer
m/Assignment04/ha4-1.py
```

```
방법 1 - 최솟값: 24, 최댓값: 97, 비교 횟수: 14
```


방법 2) $3n - 2\frac{3n}{2} - 2$ 비교가 있는 경우

이 방법은 base case일 때(element가 1개 혹은 2개일 때) 까지 배열을 반으로 계속해서 나눈다.

(Divide and Conquer방식을 사용한다)

pseudo code

```
1 procedure FindMaxMin_DC(A, i, j)
2
3 1. if i = j then
4 2.   return (A[i], A[i])
5
6 3. else if j = i + 1 then
7 4.   if A[i] < A[j] then
8 5.     return (A[i], A[j])
9 6.   else
10 7.     return (A[j], A[i])
11
12 8. else
13 9.   mid ← (i + j) / 2
14
15 10.  (minLeft, maxLeft) ← MaxMin(A, i, mid)
16
17 11.  (minRight, maxRight) ← MaxMin(A, mid + 1, j)
18
19 12.  return (min(minLeft, minRight), max(maxLeft, maxRight))
```

bash

전체적인 로직은 다음과 같다.

1. Base case (1개 element) : $(A[i], A[i])$ 을 반환한다.
2. Base case(2개 element) : 최솟값과 최댓값을 결정하기 위해 한 번의 비교를 수행한다.
3. 재귀 단계 : 두 부분의 min 값을 비교하고, 두 부분의 max 값을 비교다 (총 2번의 비교).

```

1 def findMaxMin_DC(A, i, j):
2     if i == j: # Base case: 1 element
3         return A[i], A[i]
4     elif j == i + 1: # Base case: 2 elements
5         return (A[i], A[j]) if A[i] < A[j] else (A[j], A[i])
6     else:
7         mid = (i + j) // 2
8         min1, max1 = findMaxMin_DC(A, i, mid)
9         min2, max2 = findMaxMin_DC(A, mid + 1, j)
10        return min(min1, min2), max(max1, max2)

```

python

$n = 8$ 일 때, $\frac{3(8)}{2} - 2 = 12 - 2 = 10$ 번의 비교가 이루어졌다.

//비교횟수 추가한 버전

```

1 A = [24, 75, 92, 83, 61, 48, 97, 50]
2
3 # 방법 2
4 def findMaxMin_2(A, i, j):
5     count = 0
6     if i == j: # 요소 1개
7         return A[i], A[i], 0
8
9     elif j == i + 1: # 요소 2개
10        count += 1
11        if A[i] < A[j]:
12            return (A[i], A[j], count)
13        else:
14            return (A[j], A[i], count)
15    else:
16        mid = (i + j) // 2
17        (min1, max1, c1) = findMaxMin_2(A, i, mid)
18        (min2, max2, c2) = findMaxMin_2(A, mid + 1, j)
19
20        # 합칠 때 발생하는 비교 2회
21        count = c1 + c2 + 2
22        return min(min1, min2), max(max1, max2), count
23
24 min_val, max_val, count2 = findMaxMin_2(A, 0, len(A) - 1)

```

```

24 min_val_2, max_val_2, count2 = findMinMaxMin_2(A, 0, len(A) - 1)
25 print(f"방법 2 - 최솟값: {min_val_2}, 최댓값: {max_val_2}, 비교 횟수: {count2}")

```

python

82106@DESKTOP-U8T6R2I MINGW64 ~/develop

```

$ C:/Users/82106/AppData/Local/Programs/Python/Python314/
python.exe c:/Users/82106/develop/2026-ComputerAlgorithm/Assignment04/ha4-1.py

```

방법 2 - 최솟값: 24, 최댓값: 97, 비교 횟수: 10

Step2:

Analyze the time complexities of your implementations

- Sufficient details have to be provided based on your code and time complexity

(This complexity analysis just shows the number of comparisons based on your code, NOT use recurrence relations and asymptotic bound)

82106@DESKTOP-U8T6R2I MINGW64 ~/develop

```

$ C:/Users/82106/AppData/Local/Programs/Python/Python314/python.exe c:/Users/82106/develop/2026-ComputerAlgorithm/Assignment04/ha4-1.py

```

방법 1 - 최솟값: 24, 최댓값: 97, 비교 횟수: 14

방법 2 - 최솟값: 24, 최댓값: 97, 비교 횟수: 10

- **방법 1의 경우**, 이 알고리즘은 배열의 요소를 하나씩 순회하며 최댓값과 최솟값을 갱신한다.
 - **비교 방식**: 각 단계마다 $A[i] > \text{max_val}$ 비교 1회, $A[i] < \text{min_val}$ 비교 1회를 독립적으로 수행
 - 첫 번째 요소를 제외한 나머지 $n - 1$ 개의 요소에 대해 각각 2번의 비교가 수행.
 - 총 비교 횟수 $C(n) = 2 \times (n - 1) = 2n - 2$
 - $C(8) = 14$
- **방법 2의 경우**, 이 알고리즘은 배열을 반으로 나누어 부분 문제의 결과를 얻은 뒤, 최종적으로 그 결과들끼리 다시 비교한다.
 - **Base Case**: 요소가 2개인 경우 1번의 비교를 수행
 - **Recursive Step**: 두 개의 부분 문제에서 반환된 $(\text{min1}, \text{max1})$ 과 $(\text{min2}, \text{max2})$ 를 합칠 때, 최솟값끼리 1

회, 최댓값끼리 1회, 총 2번의 비교를 추가로 수행

- 배열 크기 $n = 8$ 일 때의 비교 횟수 흐름:
 - 길이 8인 배열을 길이 4인 두 그룹으로 나눔 (결과 합칠 때 **2회** 비교)
 - 길이 4인 그룹을 길이 2인 그룹으로 나눔 (각각 **2회**씩 총 4회 비교)
 - 길이 2인 그룹에서 최솟값/최댓값을 구함 (각각 **1회**씩 총 4회 비교)
- 총 합계: $2 + 4 + 4 = 10$ 번의 비교 수행.
- $C(n) = \frac{3n}{2} - 2$
- $C(8) = 10$

* 입력 크기 n 인 배열에 대해 수행되는 비교 횟수를 $C(n)$ 으로 정의한다.

분석 항목	방법 1 (Linear)	방법 2 (Divide & Conquer)
비교 횟수	$2n - 2$	$\frac{3n}{2} - 2$
$n = 8$ 일 때 결과	14회	10회
효율성	낮음	높음

Step3:

Explain why one type performs more efficiently than the other.

방법 2의 경우가 (분할정복 방법, **divide and conquer**) element들을 pair로 묶음으로써 비교 횟수를 줄여주기 때문에 더 효율적이다.

방법 1의 경우, 모든 element를 현재의 최댓값 및 최솟값과 비교한다.

반면, 분할정복 방식(방법 2)에서는 먼저 각 pair 내의 element들을 비교한 다음, 그 결과 중 더 큰 값은 현재의 최댓값과만 비교하고, 더 작은 값은 현재의 최솟값과만 비교한다. 이 방식을 통해, 먼저 작은 부분부터 요소들을 필터링한 후에야 전체 최댓값/최솟값과 비교하므로, 전체 연산 횟수가 크게 줄어든다. 데이터 n 이 커질수록 두 방법의 비교 횟수 차이는 $n/2$ 만큼 계속 벌어지게 된다.

(14-10 = 4 번의 비교를 덜 하므로, 약 28.6% 더 적은 비교를 진행한다.)



sihyes

24학번 컴퓨터공학과



sihyes

내용을 입력하세요.

